Instituto Tecnológico y de Estudios Superiores de Monterrey

Brain MRI Segmentation

Jesus Ramirez Delgado

Development of Advanced Computer Science Applications

Abstract

1. Introduction

2. Data Exploration

2.1 Dataset Introduction

The dataset source comes from "*Brain MRI segmentation*" by Mateusz Buda [1]. The dataset in turn is the LGG Segmentation Dataset, which focuses on the segmentation of abnormal regions in brain MRI images from patients diagnosed with lower-grade glioma (LGG). This dataset was built from medical imaging data obtained from The Cancer Imaging Archive (TCIA) and is linked to cases from The Cancer Genome Atlas (TCGA) lower-grade glioma collection. In addition to imaging data, the dataset also includes clinical and genomic information for each patient through the file data.csv.

This dataset is designed for the study of brain tumor abnormalities visible in FLAIR MRI sequences, and it is commonly used in medical image segmentation and radiogenomic analysis. The segmentation masks were created manually and serve as the ground truth for supervised learning tasks.

The dataset is especially relevant because it combines two important sources of information:

1. Medical imaging data, which supports the segmentation task.
2. Patient-level genomic and clinical data, which can be used for complementary analysis and future radiogenomic studies.

2.2 Structure of the Dataset

The dataset is organized into **110 folders**, each corresponding to one patient. Every folder contains multiple MRI slices and their corresponding segmentation masks. The files follow a naming convention that allows each MRI image to be matched directly with its mask.

The general naming format is:

TCGA_<institution-code>_<patient-id>_<slice-number>.tif

and the corresponding mask is:

TCGA_<institution-code>_<patient-id>_<slice-number>_mask.tif

This means that for each MRI slice there is an associated mask file with the same identifier and the suffix _mask.

2.3 File Types and Format

The images are stored in .tif format and each image has 3 channels. These channels correspond to different MRI sequences:

- pre-contrast
- FLAIR
- post-contrast

For some patients, one of these sequences is missing. In those cases, the missing sequence was replaced with the FLAIR sequence so that all images maintain a consistent 3-channel structure.

The segmentation masks are also stored in .tif format, but they are single-channel binary images. These masks indicate whether each pixel belongs to the abnormal region visible in the FLAIR sequence. In the explored dataset, the mask values are represented as 0 for background and 255 for abnormal region, which can be converted into 0 and 1 for training purposes.

2.4 Number of Patients and Samples (instances)

The dataset contains information for 110 patients. From the exploratory analysis, the dataset includes:

- 3929 MRI images
- 3929 segmentation masks

All MRI images have shape:

- $256 \times 256 \times 3$

All masks have shape:

- 256×256

Thus, the dataset is composed of 2D MRI slices rather than full 3D volumes, and each slice is paired with its corresponding segmentation mask.

2.5 Additional Information

Besides the image and mask files, the dataset includes a file called data.csv, which contains patient-level metadata. This file provides information such as:

- **Genomic subtype clusters:** molecular groupings of tumors based on genomic data, used to identify biologically similar glioma subtypes.
- **Histological type:** tissue-based classification describing the tumor according to its microscopic characteristics.
- **Tumor grade:** indicates the severity or aggressiveness of the tumor.
- **Tumor location:** anatomical region of the brain where the tumor is found.
- **Gender**: biological sex of the patient.
- **Age:** patient age at the time of diagnosis or pathological evaluation.

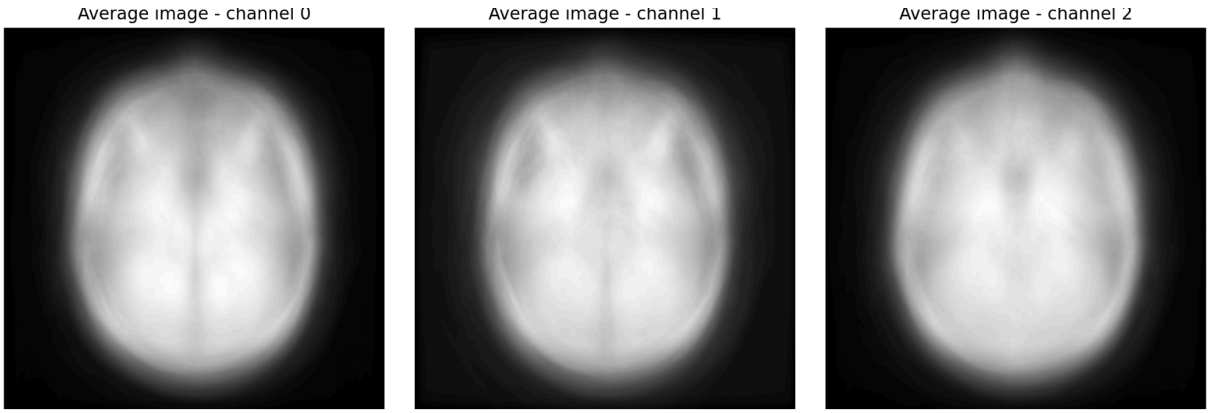
- **Race:** demographic category reported for the patient.
- **Ethnicity:** ethnic background reported for the patient.
- **Survival-related information:** variables associated with patient outcome, such as survival status or death indicator.

This metadata is not the direct target of the segmentation problem, but it is useful for understanding the patient and supporting complementary analyses.

2.6 Exploration

2.6.1 Average Image

The following image is a recreation of the average image of the dataset for the three channels.

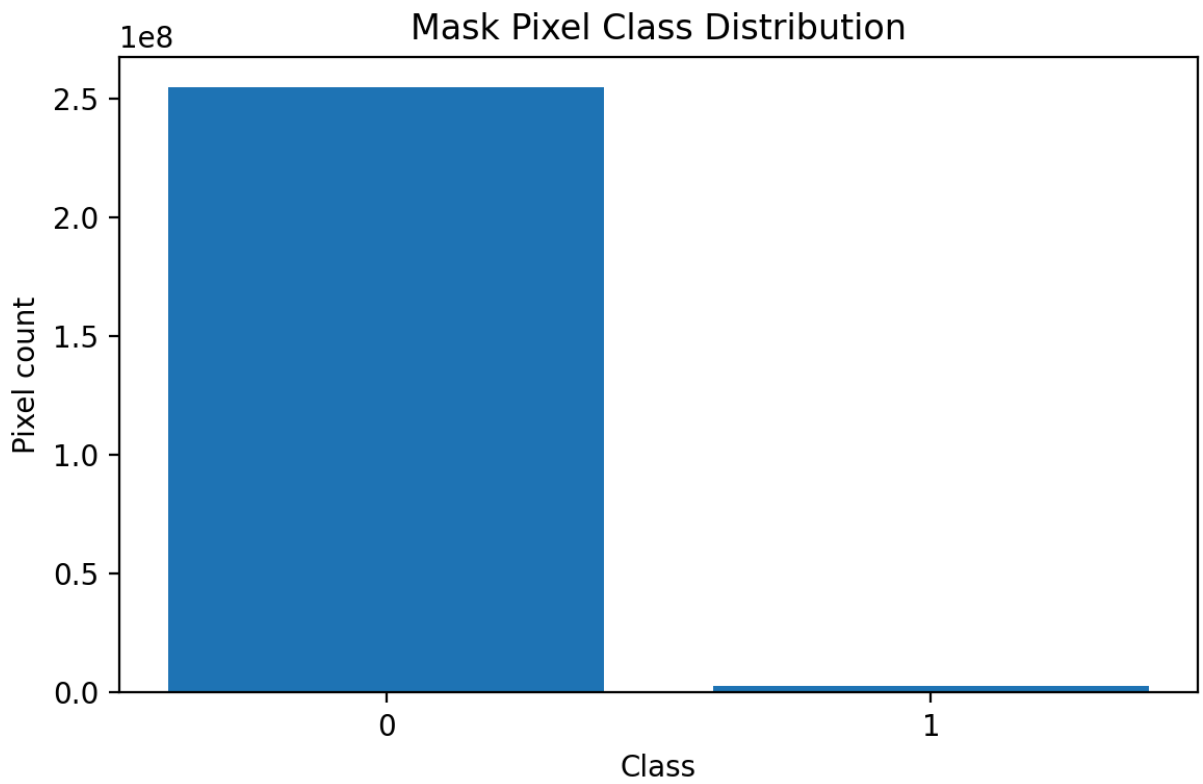


2.6.2 Class distribution

For the image segmentation task, the labels are provided through the segmentation masks, where each pixel belongs to one of two classes.

- **Class 0:** Represents the background, meaning pixels that do not correspond to the abnormal region and therefore are not the target of interest.
- **Class 1:** Represents the FLAIR abnormality region, which is the area that the model is expected to detect and segment.

Pixel class distribution		
Class	Number of pixels	Percentage
0	254,839,090	98.97%
1	2,651,854	1.03%



We can note that the dataset is far from being balanced, this due the nature of the dataset and difficult to get new instances.

2.7 Data quality

The quality of the dataset is appropriate for a medical image segmentation task, since it provides a consistent structure across samples and includes manually annotated masks that serve as reliable ground truth. All MRI slices have the same spatial resolution of 256 × 256 pixels, and the images are stored in a uniform 3-channel .tif format, which facilitates preprocessing and model training. In addition, each image has a corresponding segmentation mask, allowing a direct pairing between input data and target labels. However, the dataset also presents some challenges that must be considered during analysis, such as the strong class imbalance.

3.0 Data Preparation

3.1 Data split

To split the dataset, a patient-level strategy was selected in order to avoid data leakage. Since each patient contains multiple MRI slices, all images and masks from the same patient must remain in the same subset.

First, a final test set is reserved using approximately 10–15% of the 110 patients. The remaining patients are then used in a 5-fold cross-validation scheme. In each fold, 4 parts are used for training and 1 part is used for validation. This approach allows the model to be validated several times while keeping an independent final test set for the final evaluation.

Stage	Distribution	Approximate patients	Purpose
Final test set	10–15%	11–17	Final unbiased evaluation
Remaining data	85–90%	93–99	Used for cross-validation
Training in each fold	4/5 of remaining data	about 74–79	Model training
Validation in each fold	1/5 of remaining data	about 19–20	Model validation

3.2 Data augmentation

In order to improve the robustness of our data set, the next data augmentation modifications will be applied for the training data, with

Data augmentation	Description / purpose
Horizontal flip	Introduces spatial variation in the image orientation and helps the model generalize better.
Small rotations	Simulates slight changes in image orientation during acquisition while preserving the anatomical structure.
Small shifts / translations	Moves the image slightly in horizontal or vertical directions to improve robustness to position changes.
Small scaling / zoom	Helps the model adapt to small variations in the apparent size of the abnormal region.
Brightness adjustment	Simulates differences in image intensity between scans.
Contrast adjustment	Improves robustness to variations in tissue contrast and scanner conditions.

The selected augmentations are not intended to be applied aggressively. Instead, they are used with small magnitudes and controlled probability. In this way, augmentation increases variability in the training data without introducing unrealistic distortions.

3.3 Prepared dataset

After the data augmentation applied to training data and the separation into folds, the ramming processed data follows the next structure:

Stage / Fold	Subset	Patients	Image-mask pairs
Final split	Train + Validation pool	99	3599
	Final Test	11	330
Fold 1	Train	79	2809
	Validation	20	790
Fold 2	Train	79	2784
	Validation	20	815
Fold 3	Train	79	2926
	Validation	20	673
Fold 4	Train	79	2942
	Validation	20	657
Fold 5	Train	80	2935
	Validation	19	664

4.0 Modeling

The modeling phase was divided into N iterations, where each one has a different purpose in the project, since they have a baseline to compare the architecture proposals and the improvement of those depending on the results of each iteration. This approach is pretended to improve the model and get better results.

4.1 Threshold-based segmentation

A segmentation threshold was selected, a very basic baseline that is not pretended to get accurate results, and is a start point for comparison against the next models. Threshold segmentation is an appropriate

baseline for image segmentation because it is a simple, interpretable, and computationally inexpensive method that directly separates foreground from background based on pixel intensity values. The disadvantage of this approach is that it does not learn patterns or spatial context from data.

Note: Since this approach is not a trainable model, there is not weights saved, instead the configuration was saved as a JSON, the information can be consulted on the repostiry,

4.1.1 Parameters and configuration

Aspect	**Configuration**
Model name	threshold_baseline
Model type	Classical non-trainable baseline
Segmentation approach	Fixed threshold segmentation
Input image type	Multi-channel .tif image
Channel used for segmentation	Channel index 1
Channel name used	FLAIR
Normalization	Min-max normalization per image
Threshold value	0.55
Output	Binary mask
Output classes	0 = background, 1 = lesion
Training required	No
Learned weights	No
Cross-validation folds	5
Validation strategy	Fold-based validation on fold 1 to fold 5
Final evaluation	Separate final test split

4.2 Encoder - decoder V1

The architecture selected for a more solid model was an encoder decoder, this because it is specifically designed to solve the two main requirements of the task: understanding the content of the image and recovering the spatial location of each region at pixel level. The encoder progressively transforms the input image into a compact set of high-level feature representations, allowing the model to capture relevant patterns, textures, and contextual information. The decoder then upsamples these learned representations to recover the spatial resolution needed to generate a pixel-wise segmentation mask.

Despite the advantages, this architecture represents some limitations to consider such as the loss of spatial detail during downsampling, possible coarse boundary reconstruction, higher computational cost, sensitivity to hyperparameters, and risk of overfitting on limited datasets.

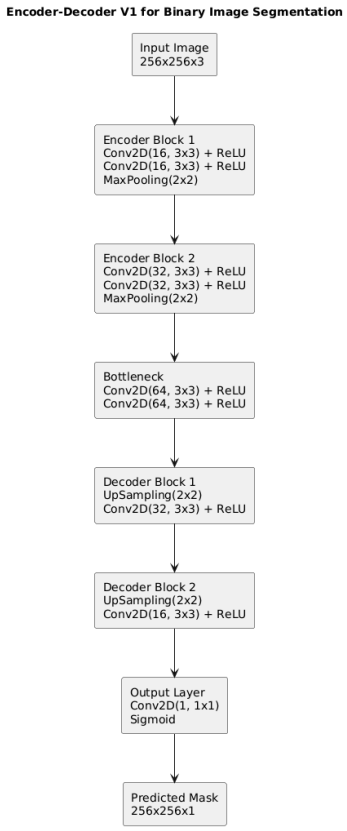
4.2.1 Parameters and configuration

Aspect	**Configuration**
Model name	encoder_decoder_v1
Framework	Keras / TensorFlow
Task	Binary image segmentation
Input shape	256 x 256 x 3
Output shape	256 x 256 x 1
Output activation	Sigmoid
Loss function	Binary Cross-Entropy
Optimizer	Adam
Learning rate	1e-3
Batch size	8
Epochs	15
Prediction threshold	0.5
Cross-validation folds	5
Random seed	42
Image normalization	Per-channel min-max normalization

Checkpoint strategy	Best model saved by val_loss

4.2.2 Architecture

This first version has a basic configuration with two main encoding and decoding blocks, using convolutional layers followed by ReLU activation functions to extract nonlinear features from the input image. In the encoder, max-pooling layers are applied to progressively reduce the spatial dimensions and capture higher-level contextual information. In the decoder, upsampling layers are used to restore the spatial resolution and reconstruct the segmentation mask. Finally, a 1x1 convolution with sigmoid activation produces a pixel-wise binary output, where each pixel is classified as background or lesion. This configuration represents a basic standard for modern encoder-decoder architectures and leaves room for possible improvements,



4.3 Encoder - decoder V2

4.3.1 Parameters and configuration

4.3.2 Architecture

5.0 Evaluation

The evaluation section aims to measure the segmentation performance of each model in a consistent and comparable way. Since all approaches in this study produce a predicted segmentation mask, the same evaluation metrics can be applied across classical and deep learning models. To assess performance, the metrics selected were Intersection over Union (IoU), Precision, Recall, and Pixel Accuracy.

- **IoU:** The primary metric because it directly measures the agreement between predicted and ground-truth masks.
- **Recall:** Also highly relevant, especially in lesion segmentation, where missing true lesion pixels is undesirable.
- **Precision:** Helps identify over-segmentation.
- **Pixel Accuracy:** Reported only as a complementary metric due to the strong class imbalance.

5.1 Threshold-based segmentation

5.1.1 Results

Fold / Split	TP	FP	TN	FN	IoU	Precision	Recall	Pixel Accuracy
Fold 1	122,657	1,611,309	49,589,050	450,424	0.0562	0.0707	0.2140	0.9602
Fold 2	88,692	1,816,349	51,072,561	434,238	0.0379	0.0466	0.1696	0.9579
Fold 3	53,812	1,442,680	42,344,552	264,684	0.0306	0.0360	0.1690	0.9613
Fold 4	127,653	1,705,901	40,820,137	403,461	0.0571	0.0696	0.2404	0.9510
Fold 5	127,238	1,872,837	41,166,491	349,338	0.0542	0.0636	0.2670	0.9489
Global CV	520,052	8,449,076	224,992,791	1,902,145	0.0478	0.0580	0.2147	0.9561
Final Test	61,434	681,908	20,715,315	168,223	0.0674	0.0826	0.2675	0.9607

5.1.2 Metrics analysis

- **IoU:** This is the most important metric in this baseline analysis because it directly measures overlap between the predicted mask and the ground truth. The obtained values are very low, which indicates poor segmentation quality.

- **Precision:** Precision is also very low, showing that a large proportion of the pixels predicted as lesions actually correspond to background. This suggests many false positives.

- **Recall:** Recall is slightly higher than Precision, meaning that the method is able to detect part of the lesion area, but still misses a large amount of true lesion pixels.

- **Pixel Accuracy:** Although Pixel Accuracy is high, it must be interpreted carefully because the dataset contains a large majority of background pixels. For that reason, this metric is less informative than IoU for this task.

5.1.3 Folds and test analysis

Overall, the behavior across folds suggests that thresholding is unstable and limited for this problem, since performance changes between folds but remains weak in all cases.

The final test slightly improved over global cross-validation, but performance remained low overall. This confirms that thresholding can serve as a basic reference baseline, yet it is not strong enough for accurate lesion segmentation due to its limited overlap and low precision.

5.1.4 Conclusions

These results confirm that threshold segmentation is useful as a simple baseline, but its low overlap and low precision demonstrate that more advanced learning-based models are necessary for this problem.

5.2 Encoder - decoder V1

5.2.1 Results

Fold / Split	TP	FP	TN	FN	IoU	Precision	Recall	Pixel Accuracy
Fold 1	255,563	129,210	51,071,149	317,518	0.3639	0.6642	0.4459	0.9914
Fold 2	313,217	75,903	52,813,007	209,713	0.5230	0.8049	0.5990	0.9947
Fold 3	218,421	79,043	43,708,189	100,075	0.5494	0.7343	0.6858	0.9959
Fold 4	357,662	103,124	42,422,914	173,452	0.5639	0.7762	0.6734	0.9936
Fold 5	225,841	22,261	43,017,067	250,735	0.4527	0.9103	0.4739	0.9937

Global CV	1,370,704	409,541	233,032,326	1,051,493	0.4841	0.7700	0.5659	0.9938
Final Test	175,608	80,671	21,316,552	54,049	0.5659	0.6852	0.7647	0.9938

5.2.2 Metrics analysis

- **IoU:** IoU shows a clear improvement compared with the threshold baseline, indicating that the encoder-decoder achieves a much stronger overlap between predicted masks and ground truth.

- **Precision:** Precision is high in most folds, which means that a large proportion of the predicted lesion pixels are correct.

- **Recall:** Recall is also considerably higher than in the threshold baseline, showing that the model is able to recover a larger portion of the true lesion area.

- **Pixel Accuracy:** Pixel Accuracy remains very high, but as in the previous case, it should be interpreted carefully because background pixels dominate the images.

5.2.3 Folds and test analysis

The encoder-decoder shows substantially better and more consistent performance than the threshold baseline across all folds. Folds 3 and 4 achieved the strongest balance between Precision and Recall, resulting in the highest IoU values. Overall, the fold results show that the encoder-decoder is capable of learning meaningful lesion patterns, although some variability between folds is still present.

The combination of a higher Recall and solid IoU in test suggests that the model produced a more complete lesion segmentation on the final test set. This confirms that the encoder-decoder provides a much more effective segmentation approach than the threshold baseline and establishes a solid first deep learning benchmark.

5.2.4 Conclusions

These results show that the encoder-decoder significantly outperforms the threshold baseline and provides a strong initial deep learning solution for lesion segmentation. However this model can get a better performance and according to the previous results the work to be done in order to increase the performance of our model but not significantly considering the computational cost.

5.3 Encoder - decoder V2

5.3.1 Results

5.3.2 Metrics analysis

5.3.3 Folds and test analysis

5.3.4 Conclusions

6.0 General Conclusions

7.0 Future Work

References

[1] M. Buda, “Brain MRI segmentation,” *Kaggle*, [Online]. Available: https://www.kaggle.com/datasets/mateuszbuda/lgg-mri-segmentation/data. [Accessed: Apr. 12, 2026].

[2] K. Alomar, H. I. Aysel, and X. Cai, “Data augmentation in classification and segmentation: A survey and new strategies,” *Journal of Imaging*, vol. 9, no. 2, p. 46, 2023. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC9966095/. [Accessed: Apr. 12, 2026].

[3] GeeksforGeeks, “Thresholding-Based Image Segmentation,” *GeeksforGeeks*. [Online]. Available: https://www.geeksforgeeks.org/software-engineering/thresholding-based-image-segmentation/. [Accessed: Apr. 15, 2026].

[4] GeeksforGeeks, “SegNet - A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation,” *GeeksforGeeks*. [Online]. Available: https://www.geeksforgeeks.org/computer-vision/segnet-a-deep-convolutional-encoder-decoder-architecture-for-image-segmentation/. [Accessed: Apr. 17, 2026].